LE Hermetic Security Audit 20260701

LE Hermetic Security Audit 20260701

Let's Encrypt Hermetic Infra + Proxy Lanes — Security Audit

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Scope:** `deploy/letsencrypt/**` (hermetic ACME stack) + committed proxy Challenge lanes (`challenges/scripts/proxy_*.sh`, `challenges/scripts/run_proxy_challenges.sh`) + `tools/helixqa/**` **Method:** READ-ONLY static review (Read + grep) — no code changed, no container booted, no operator resource touched **Authority:** Helix Constitution §11.4.169 (security test-type), §11.4.10 (credentials), §11.4.6 (no-guessing — every finding carries `file:line` evidence or is marked UNCONFIRMED), §11.4.44 (revision header)

Verdict summary

Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	1
LOW	3
INFO (positive/hardening)	6

No real secret leaked. No token, private key, password, API key, or credential value was found in any tracked OR untracked file in scope (see Secret-scan result below). The one committed cryptographic artifact (`pebble-ca/pebble.minica.pem`) is a PUBLIC test CA certificate with no private-key material, and it is gitignored.

Tracking status (context): the entire `deploy/letsencrypt/` tree is currently UNTRACKED (`git ls-files deploy/letsencrypt/` returns empty; matches session `?? deploy/`). The proxy Challenge lanes and `tools/helixqa/**` ARE tracked. Findings below apply regardless of tracking state (they gate a future commit + boot).

Secret-scan result (§11.4.10)

Explicit scan across the full audit scope (tracked + untracked):

- Value-bearing `password|secret|token|api_key|bearer|authorization` assignments — **none found** (only NAME-ONLY references such as `ACME_DNS_TOKEN_SECRET_NAME=helixproxy_caddy_dns_token`, a Podman-secret NAME, never a value).
- `BEGIN … PRIVATE KEY` / `BEGIN RSA` / `BEGIN OPENSSH` markers — **none** anywhere in scope.
- Vendor token shapes (`AKIA…`, `ghp_…`, `xox[baprs]-…`, `sk-…`) — **none**.
- `deploy/letsencrypt/pebble-ca/pebble.minica.pem` — `-----BEGIN CERTIFICATE-----` only, **no `PRIVATE KEY` block**; `git check-ignore` confirms it is ignored via `.gitignore:32 pebble-ca/`.

Conclusion: clean. No credential exposure.

MEDIUM findings

M1 — Caddy HTTPS/HTTP host ports published on ALL interfaces (0.0.0.0) in the hermetic stack

Evidence: `deploy/letsencrypt/compose.hermetic.yml:244-245`

```
- "${CADDY_HTTPS_PORT:-8443}:443"   # HTTPS (TLS termination)
- "${CADDY_HTTP_PORT:-8080}:80"     # HTTP -> HTTPS redirect only
```

Neither caddy port carries a `127.0.0.1:` bind prefix, so Podman publishes them on `0.0.0.0` (every host interface). By contrast the two supporting services are correctly loopback-scoped: `pebble` at `compose.hermetic.yml:106-107` (`127.0.0.1:…:14000/15000`) and `challtestsrv` at `:139` (`127.0.0.1:…:8055`).

Failure scenario: when the hermetic stack is booted on a workstation attached to a LAN/Wi-Fi (the operator's own §11.4.174 shared-host case), any other host on that network can reach `https://<host-ip>:8443` and `http://<host-ip>:8080`. That front terminates TLS and `reverse_proxy`es to the pod-internal upstream (`CADDY_UPSTREAM`, default `admin:80`), so a LAN peer can drive the test upstream through the exposed Caddy. For a purely hermetic self-issuance test this is broader exposure than required.

Nuance (not a defect in the production TLS-front role): Caddy IS the public TLS terminator by design (`Caddyfile:11-15`), so a network-facing bind is intentional for the real staging/prod path. The finding is that the HERMETIC test profile inherits that wide bind rather than loopback. **Recommended (advisory, not applied — READ-ONLY):** default the hermetic caddy publishes to `127.0.0.1:${CADDY_HTTPS_PORT}:443` / `127.0.0.1:${CADDY_HTTP_PORT}:80`, and let the operator widen the bind explicitly for the network-facing cutover. Ranked MEDIUM because exposure is real but bounded (a test upstream, no secret behind it) and conditional on a non-isolated host.

LOW findings

L1 — Base + service images pinned by TAG, not by `@sha256` digest (supply chain)

Evidence: `deploy/letsencrypt/Dockerfile.caddy:28` (`FROM docker.io/library/caddy:${CADDY_VERSION}-builder`), `:53` (`…-alpine`); `compose.hermetic.yml:78` (`ghcr.io/letsencrypt/pebble:2.6.0`), `:125` (`pebble-challtestsrv:2.6.0`), `:165` (`coredns/coredns:1.11.1`).

Failure scenario: a mutable tag can be re-pointed upstream (registry compromise or a maintainer force-repush of a tag), so a later `podman build`/`pull` could fetch different bytes than were reviewed. **Mitigating fact:** the files self-acknowledge this as intended-future work (`compose.hermetic.yml:75-77, :123-124` — "PENDING boot-verify … pin the @sha256 digest") per §11.4.6/§11.4.173, so it is a tracked gap, not an oversight. Ranked LOW.

L2 — challtestsrv DNS provider has no runtime hermetic guard (documentation-only control)

Evidence: `deploy/letsencrypt/caddy-challtestsrv-dns/provider.go:10-13` ("HERMETIC-TEST-ONLY … Do NOT point this provider at any real DNS zone") is a doc comment; `Provision` (`:73-83`) and `post` (`:155-175`) apply no scheme/host allowlist and no "refuse non-hermetic target" check.

Failure scenario (bounded, near-structurally-impossible): if an operator mis-set `CHALLTESTSRV_MGMT_URL` to an arbitrary reachable URL, the module would `POST /set-txt` / `/clear-txt` there. **Why the blast radius is near-zero:** the module only speaks challtestsrv's bespoke `POST /set-txt` management API; a real DNS provider exposes no such endpoint, so it cannot modify real DNS records — the worst realistic outcome is a non-2xx error against an unrelated host. Ranked LOW (defense-in-depth suggestion: validate the URL is `http(s)` and optionally warn if the host is non-loopback / non-pod-internal). No injection risk exists on the call itself — see INFO-3.

L3 — CoreDNS forward target is a hardcoded IP placeholder rewritten at boot (§11.4.111 by-address binding)

Evidence: `deploy/letsencrypt/coredns/Corefile:7` (`forward . 10.89.4.2:8053`). The compose comment (`compose.hermetic.yml:158-160`) states the line is "(re)written with challtestsrv's live IP by `phase3_hermetic_issue.sh` before this service starts."

Failure scenario: the tracked Corefile ships a fixed pod-subnet IP that is only correct after a boot-time rewrite; a stale/checked-in IP that no longer matches the live challtestsrv would silently forward ACME `_acme-challenge` TXT queries to the wrong address (a §11.4.111 resolve-by-index fragility, not a confidentiality/integrity breach — the data is a public ACME challenge token). Ranked LOW; noted for §11.4.111 hygiene. The rewrite-at-boot pattern also means a script mutates a tracked config file — acceptable here (regenerated, non-secret) but worth an INFO flag for reviewers.

INFO — positive controls confirmed (hardening that is present and correct)

- **INFO-1 — Non-root container + minimal capability, cleaned up.** `Dockerfile.caddy:76` (`USER caddy`), `:68` (`setcap 'cap_net_bind_service=+ep'` — the single minimal cap to bind :80/:443, nothing broader), `:69` (`apk del libcap` — the setcap tool is removed post-use). Confirms §11.4.161 rootless + defense-in-depth.
- **INFO-2 — No privileged / host-network / dangerous mount anywhere.** Grep across `compose.hermetic.yml` for `privileged:|cap_add|network_mode|/var/run/docker.sock|/run/podman|SYS_ADMIN|pid:host|ipc:host` returns **none**. Every service is on the private `letsencrypt-hermetic` network; all config/CA mounts are read-only (`:ro,Z` — SELinux-relabelled) at `:198`

(Caddyfile), :207 (pebble-ca), and coredns Corefile (:171, read-only). Named volumes only for regenerable ACME state.

- **INFO-3 — DNS provider HTTP calls are injection-safe and bounded.** provider.go:156 marshals the payload with json.Marshal (proper escaping — no body injection); :160 builds the URL as p.ManagementURL + path where path is a hardcoded literal (/set-txt / /clear-txt, :122/:134) and the host/value come from Caddy's own ACME config, not attacker-controlled runtime input (no URL/command injection); :80 sets a 10s client timeout (no hang); :171 reads at most 512 bytes of any error body via io.LimitReader (bounded diagnostic, no unbounded/echo leak). No os/exec, no shell, no template concatenation into a command.
- **INFO-4 — Secrets handled by NAME + PATH only, and commented off for hermetic.** compose.hermetic.yml:47-49 (top-level secrets: block commented), :225-226 (caddy secrets: mount commented), :190 (ACME_DNS_TOKEN_FILE is a mount PATH, value read at runtime), .env.example:48-58 (NAME-only, explicit "NEVER the value"). Admin API OFF by default: compose.hermetic.yml:215 (CADDY_ADMIN=${…:-off}), Caddyfile:44 (admin {$CADDY_ADMIN:off}), admin port 2019 commented at :247 and loopback-only when enabled. Fully §11.4.10-compliant.
- **INFO-5 — .gitignore is comprehensive and effective.** deploy/letsencrypt/.gitignore excludes .env/.env.* (:13-14, allowing .env.example), *.key/*.pem/*.crt/*.csr/*.p12/*.pfx (:20-25), data/+caddy-*/ (:26-29), pebble-ca/ (:32), secrets/ +*.secret+*_token (:35-37). Runtime crypto material and the materialized public CA cannot be committed. Proxy-lane evidence lands under qa-results/ which is gitignored at root (.gitignore:51).
- **INFO-6 — Proxy Challenge lanes are read-only clients with no credential-leaking logs.** proxy_forward_http_challenge.sh, proxy_socks5_challenge.sh, proxy_cache_challenge.sh and run_proxy_challenges.sh invoke only curl through the proxy with quoted args (no shell injection), never start/stop/reconfigure a container, and capture response headers via a grep allowlist restricted to HTTP/|Via|Cache-Control|Age|Server|Content-Type|X-Cache|Expires (proxy_forward_http_challenge.sh:107, proxy_cache_challenge.sh:135) — Authorization/Cookie/Set-Cookie are NOT captured. run_proxy_bank.sh runs the HelixQA subprocess under a clean env -i (:178) and only reads (cat) the project's own proxy-squid access log — no secret written to any log path; evidence dirs are created with the default umask (not world-writable) under gitignored qa-results/.

Coverage note

Every file named in the task was read in full: `provider.go`, `compose.hermetic.yml`, `Dockerfile.caddy`, `Caddyfile`, `build.sh`, `.env.example`, `.gitignore`, `coredns/Corefile`, `pebble-ca/pebble.minica.pem` (header + private-key scan), the three `proxy_*_challenge.sh` lanes, `run_proxy_challenges.sh`, `tools/helixqa/runner/run_proxy_bank.sh`, `tools/helixqa/banks/proxy.yaml` + `routes/proxy_cache.yaml`, and the module README. The HelixQA bank YAMLs are declarative test definitions with no secrets and no executable injection surface. Nothing in scope was left unreviewed; no finding is marked UNCONFIRMED.